



FORMATO PARA ELABORACIÓN DE TALLERES ESTUDIANTILES

Departamento:

CIENCIAS DE LA COMPUTACION

Carrera: ELECTRONICA Y AUTOMATIZACION

Asignatura: FUNDAMENTOS DE PROGRAMACION
NRC: 28561

Apellidos y nombres del estudiante: Francisco Comina

1. Antecedentes

Esta guía orienta al estudiante para analizar, ejecutar y modificar ejercicios de funciones en C/C++ usando Code::Blocks. Los ejercicios trabajan funciones void, funciones con return, parámetros, uso de banderas, menú, validación de división para cero, raíz cuadrada y potencia.

2. Objetivos

Aplicar el uso de funciones en lenguaje C para organizar programas mediante parámetros, retorno de valores y validación de datos, desarrollando soluciones más claras, reutilizables y eficientes.

3. Desarrollo

Ejercicio 57: Menú, mayor y valor absoluto

Objetivo: Aplicar funciones dentro de un menú para calcular el mayor de dos números y el valor absoluto.

```
#include <stdio.h>
#include <stdlib.h>
int n1, n2;
void ingresar();
void mayor();
void absoluto();

int main()
{
    int opcion;

    do
    {
        printf("\n===== \n");
        printf("    MENU \n");
        printf("===== \n");
        printf("1. Numero mayor \n");
        printf("2. Valor absoluto \n");
        printf("3. Salir \n");
        printf("Seleccione una opcion: ");
        scanf("%d", &opcion);

        if(opcion != 3)
        {
            ingresar();
        }
    }
```

```

switch(opcion)
{
    case 1:
        mayor();
        break;

    case 2:
        absoluto();
        break;

    case 3:
        printf("\nPrograma finalizado.\n");
        break;

    default:
        printf("\nOpcion invalida.\n");
}

}while(opcion != 3);

return 0;
}
void ingresar()
{
    printf("\nIngresa el primer numero: ");
    scanf("%d",&n1);

    printf("Ingresa el segundo numero: ");
    scanf("%d",&n2);
}
void mayor()
{
    if(n1 > n2)
        printf("\nEl numero mayor es: %d\n", n1);
    else if(n2 > n1)
        printf("\nEl numero mayor es: %d\n", n2);
    else
        printf("\nLos dos numeros son iguales.\n");
}
void absoluto()
{
    int abs1, abs2;

    if(n1 < 0)
        abs1 = -n1;
    else
        abs1 = n1;

    if(n2 < 0)
        abs2 = -n2;
    else

```

```
C:\Users\LABS-ESPE\Documen X + v
2. Valor absoluto
3. Salir
Seleccione una opcion: 1

Ingrese el primer numero: 45
Ingrese el segundo numero: 123

El numero mayor es: 123

=====
MENU
=====
1. Numero mayor
2. Valor absoluto
3. Salir
Seleccione una opcion: 2

Ingrese el primer numero: 45
Ingrese el segundo numero: 321

Valor absoluto de 45 = 45
Valor absoluto de 321 = 321

=====
MENU
=====
1. Numero mayor
2. Valor absoluto
3. Salir
Seleccione una opcion: 3
```

abs2 =
n2;

```
printf("\nValor absoluto de %d = %d\n", n1, abs1);
printf("Valor absoluto de %d = %d\n", n2, abs2);
}
```

Ejecución

Pregunta de análisis: ¿Qué ventaja tiene usar un menú cuando el programa ofrece más de una operación? Al nosotros usar un menú se permite que el usuario elija la operación que desea realizar sin tener que volver a estar ejecutando el programa. Y además, facilita la organización del código.

Ejercicio 58: Funciones con return y raíz cuadrada

Objetivo: Diferenciar funciones que retornan valores enteros y reales, usando suma y raíz cuadrada

```
#include <stdio.h>
#include <math.h>
```

```
int sumar(int a, int b);
float raiz(float num);
```

```
int main()
{
    int opcion;
    int a, b;
    float num, resultado;

    do
    {
        printf("\n=====n");
        printf("MENU PRINCIPAL\n");
        printf("=====n");
        printf("1. Sumar dos numeros\n");
```

```

printf("2. Calcular raiz cuadrada\n");
printf("3. Salir\n");
printf("Seleccione una opcion: ");
scanf("%d", &opcion);

switch(opcion)
{
    case 1:

        printf("\nIngrese el primer numero: ");
        scanf("%d", &a);

        printf("Ingrese el segundo numero: ");
        scanf("%d", &b);

        printf("\nLa suma es: %d\n", sumar(a, b));

        break;

    case 2:

        printf("\nIngrese un numero: ");
        scanf("%f", &num);

        resultado = raiz(num);

        if(resultado != -1)
        {
            printf("La raiz cuadrada es: %.2f\n", resultado);
        }

        break;

    case 3:

        printf("\nPrograma finalizado.\n");

        break;

    default:

        printf("\nOpcion no valida.\n");

}

}while(opcion != 3);

return 0;
}

int sumar(int a, int b)
{

```

```

    return a + b;
}

float raiz(float num)
{
    if(num < 0)
    {
        printf("\nError: No existe raiz cuadrada de un numero negativo.\n");

```

```

*Untitled1.c  *Ejer 58.c
#include <stdio.h>
#include <math.h>

int sumar(int a, int b);
float raiz(float num);

int main()
{
    int opcion;
    int a, b;
    float num, resultado;

    do
    {
        printf("\n===== \n");
        printf("MENU PRINCIPAL\n");
        printf("===== \n");
        printf("1. Sumar dos numeros\n");
        printf("2. Calcular raiz cuadrada\n");
        printf("3. Salir\n");
        printf("Seleccione una opcion: ");
        scanf("%d", &opcion);

        switch(opcion)
        {
            case 1:
                printf("\nIngrese el primer numero: ");
                scanf("%d", &a);

                printf("Ingrese el segundo numero: ");
                scanf("%d", &b);

                printf("\nLa suma es: %d\n", sumar(a, b));

                break;

            case 2:
                printf("\nIngrese un numero: ");

```

```

"C:\Users\LABS-ESPE\Documents\
Ingrese el primer numero para sumar: 45
Ingrese el segundo numero para sumar: 87

La suma es: 132

Ingrese un numero para calcular su raiz cuadrada: 22558
La raiz cuadrada es: 150.19

Process returned 0 (0x0)   execution time : 6.001 s
Press any key to continue.

```

```

        return -1;
    }

    return sqrt(num);
}

```

Pregunta de análisis: ¿Por qué `sqrt()` necesita incluir la librería `math.h`? La función `sqrt()` para su funcionamiento necesita incluir la librería `math.h` porque forma parte de las funciones matemáticas estándar del lenguaje C.

Ejercicio 59: Parámetros y retorno de valores

Objetivo: Usar parámetros para enviar valores a una función y utilizar una función que retorna el menor de dos números.

```

#include <stdio.h>
#include <math.h>

```

```

float mayor(float a, float b);
float menor(float a, float b);

```

```

int main()
{
    float a, b;
    int op;

    do
    {
        printf("\nMENU\n");
        printf("1. Ingresar numeros\n");
        printf("2. Numero mayor\n");
        printf("3. Numero menor\n");
        printf("4. Raiz del menor\n");
        printf("5. Salir\n");
        printf("Opcion: ");
        scanf("%d",&op);

        switch(op)
        {
            case 1:
                printf("Ingrese el primer numero: ");
                scanf("%f",&a);
                printf("Ingrese el segundo numero: ");
                scanf("%f",&b);
                break;

            case 2:
                printf("Mayor = %.2f\n", mayor(a,b));
                break;

            case 3:
                printf("Menor = %.2f\n", menor(a,b));
                break;

            case 4:
                if(menor(a,b) >= 0)
                    printf("Raiz = %.2f\n", sqrt(menor(a,b)));
                else
                    printf("No existe raiz de un numero negativo.\n");
                break;

            case 5:
                printf("Fin del programa.\n");
                break;

            default:
                printf("Opcion incorrecta.\n");
                break;
        }

    }while(op != 5);

    return 0;

```

```
}
```

```
float mayor(float a, float b)
```

```
{  
    if(a > b)  
        return a;  
    return b;  
}
```

```
float menor(float a, float b)
```

The screenshot shows a C++ IDE with three tabs: *Untitled1.c, *Ejer 58.c, and *ejer59.c. The code in *ejer59.c defines two functions, `mayor` and `menor`, and a `main` function that uses a `do-while` loop to display a menu and process user input. The terminal window on the right shows the program's execution, including the menu display, input of two numbers (45.36 and 0.258), and the output of the `mayor` function (45.36).

```
1 #include <stdio.h>  
2 #include <math.h>  
3  
4 float mayor(float a, float b);  
5 float menor(float a, float b);  
6  
7 int main()  
8 {  
9     float a, b;  
10    int op;  
11  
12    do  
13    {  
14        printf("\nMENU\n");  
15        printf("1. Ingresar numeros\n");  
16        printf("2. Numero mayor\n");  
17        printf("3. Numero menor\n");  
18        printf("4. Raiz cuadrada del menor\n");  
19        printf("5. Salir\n");  
20        printf("Opcion: ");  
21        scanf("%d", &op);  
22  
23        switch(op)  
24        {  
25            case 1:  
26                printf("Ingrese el primer numero: ");  
27                scanf("%f", &a);  
28                printf("Ingrese el segundo numero: ");  
29                scanf("%f", &b);  
30                break;  
31  
32            case 2:  
33                printf("Mayor = %.2f\n", mayor(a,b));  
34                break;  
35  
36            case 3:  
37                printf("Menor = %.2f\n", menor(a,b));  
38                break;  
39  
40            case 4:  
41                printf("Raiz cuadrada del menor = %.2f\n", sqrt(b));  
42                break;  
43            case 5:  
44                break;  
45        }  
46    } while(op != 5);  
47 }
```

Terminal Output:

```
===== MENU =====  
1. Ingresar numeros  
2. Mostrar el numero mayor  
3. Mostrar el numero menor  
4. Raiz cuadrada del menor  
5. Pruebas directas  
6. Salir  
Seleccione una opcion: 1  
Ingrese el primer numero: 45.36  
Ingrese el segundo numero: 0.258  
===== MENU =====  
1. Ingresar numeros  
2. Mostrar el numero mayor  
3. Mostrar el numero menor  
4. Raiz cuadrada del menor  
5. Pruebas directas  
6. Salir  
Seleccione una opcion: |
```

```
===== MENU =====  
1. Ingresar numeros  
2. Mostrar el numero mayor  
3. Mostrar el numero menor  
4. Raiz cuadrada del menor  
5. Pruebas directas  
6. Salir  
Seleccione una opcion: 2  
  
El numero mayor es: 45.36
```

```
{  
    if(a  
< b)  
  
    return  
    a;  
  
    return  
    b;  
}
```

```
===== MENU =====  
1. Ingresar numeros  
2. Mostrar el numero mayor  
3. Mostrar el numero menor  
4. Raiz cuadrada del menor  
5. Pruebas directas  
6. Salir  
Seleccione una opcion: 3  
  
El numero menor es: 0.26  
===== MENU =====
```

```
=====
                        MENU
=====
1. Ingresar numeros
2. Mostrar el numero mayor
3. Mostrar el numero menor
4. Raiz cuadrada del menor
5. Pruebas directas
6. Salir
Seleccione una opcion: 4

La raiz cuadrada del menor es: 0.51
=====
```

```
=====
                        MENU
=====
1. Ingresar numeros
2. Mostrar el numero mayor
3. Mostrar el numero menor
4. Raiz cuadrada del menor
5. Pruebas directas
6. Salir
Seleccione una opcion: 5

Mayor(7,9) = 9.00
Mayor(0,0) = 0.00
Menor(1,9) = 1.00
=====
```

Pregunta de análisis: ¿Cuál es la diferencia entre imprimir un resultado dentro de una función y retornarlo al `main()`? La diferencia es que imprimir un resultado dentro de una función solo lo muestra en la pantalla, mientras que retornarlo al `main()` permite guardar ese resultado en una variable, para poder utilizarlo en otros cálculos o enviarlos a otras funciones. Es por ello, que al usar `return` hace que el programa sea más flexible y reutilizable.

¿Explicar por qué no se puede usar `sqrt(mayor(1,9))` con la función `mayor` actual ? No se puede usar `sqrt(mayor(1,9))` con la función `mayor` original porque esa función no nos devolvera un valor con `return`, sino que únicamente lo imprimía en pantalla

Ejercicio 60: División con validación

Objetivo: Crear una función de división que reciba dos números reales y valide división para cero.

```
#include <stdio.h>

float dividir(float a, float b);

int main()
{
    float n1, n2, r;
    int op;

    do
    {
        printf("\n===== MENU =====\n");
        printf("1. Dividir\n");
        printf("2. Salir\n");
        printf("Opcion: ");
```



```

scanf("%d",&op);

switch(op)
{
    case 1:

        printf("Ingrese el primer numero: ");
        scanf("%f",&n1);

        printf("Ingrese el segundo numero: ");
        scanf("%f",&n2);

        r = dividir(n1,n2);

        if(n2 != 0)
            printf("Resultado = %.2f\n", r);

        break;

    case 2:

        printf("Programa finalizado.\n");

        break;

    default:

        printf("Opcion incorrecta.\n");
}

}while(op != 2);

return 0;
}

float dividir(float a, float b)
{
    if(b == 0)
    {
        printf("Error: No se puede dividir para cero.\n");
        return 0;
    }

    return a / b;
}

```

```
*Untitled1.c X *Ejer 58.c X *Ejer59.c X ejercicio 60.c X
#include <stdio.h>

float dividir(float a, float b);

int main()
{
    float n1, n2, r;
    int op;

    do
    {
        printf("\n==== MENU ==== \n");
        printf("1. Dividir\n");
        printf("2. Salir\n");
        printf("Opcion: ");
        scanf("%d", &op);

        switch(op)
        {
            case 1:

                printf("Ingrese el primer numero: ");
                scanf("%f", &n1);

                printf("Ingrese el segundo numero: ");
                scanf("%f", &n2);

                r = dividir(n1, n2);

                if(n2 != 0)
                    printf("Resultado = %.2f\n", r);

                break;

            case 2:

                printf("Programa finalizado.\n");

                break;
        }
    } while (op != 2);
}
```

```
"C:\Users\LABS-ESPE\Docume X + v
==== MENU ====
1. Dividir
2. Salir
Opcion: 1
Ingrese el primer numero: 84
Ingrese el segundo numero: 28
Resultado = 3.00

==== MENU ====
1. Dividir
2. Salir
Opcion: |
```

Ejecución

Pregunta de análisis: ¿Por qué retornar 0 para indicar error puede generar confusión en una división válida? Retornar 0 para indicar un error puede generar confusión porque 0 también puede ser el resultado correcto de una división

Ejercicio 61: Potencia con parámetros

Objetivo: Aplicar una función con tres parámetros para calcular potencia al cuadrado o al cubo

```
#include <stdio.h>
#include <math.h>
```

```
float potencia(float num, int pot);
```

```
int main()
{
```

```
    float num, resultado;
    int pot, op;
```

```
    do
```

```
    {
        ===== MENU =====
        1. Dividir
        2. Salir
        Opcion: 1
        Ingrese el primer numero: 0
        Ingrese el segundo numero: 0
        Error: No se puede dividir para cero.

        ===== MENU =====
        1. Dividir
        2. Salir
        Opcion: |
```

```

printf("\n===== MENU =====\n");
printf("1. Calcular potencia\n");
printf("2. Salir\n");
printf("Opcion: ");
scanf("%d", &op);

switch(op)
{
    case 1:

        printf("Ingrese un numero: ");
        scanf("%f", &num);

        printf("Ingrese la potencia (2 o 3): ");
        scanf("%d", &pot);

        if(pot == 2 || pot == 3)
        {
            resultado = potencia(num, pot);
            printf("Resultado = %.2f\n", resultado);
        }
        else
        {
            printf("Solo se permite potencia 2 o 3.\n");
        }

        break;

    case 2:

        printf("Programa finalizado.\n");
        break;

    default:

        printf("Opcion incorrecta.\n");
    }

}while(op != 2);

return 0;
}

float potencia(float num, int pot)
{
    return pow(num, pot);
}

```

```
"C:\Users\LABS-ESPE\Docume X + v

===== MENU =====
1. Calcular potencia
2. Salir
Opcion: 1
Ingrese un numero: 56.23
Ingrese la potencia (2 o 3): 3
Resultado = 177788.73
```

```
===== MENU =====
1. Calcular potencia
2. Salir
Opcion: 1
Ingrese un numero: 4
Ingrese la potencia (2 o 3): 2
Resultado = 16.00
```

Pregunta de análisis: ¿Qué mejora permitiría que la función potencia () sea más general y reutilizable? Una mejora sería que la función potencia () permitiera calcular la potencia de cualquier número con cualquier exponente, en lugar de limitarse solo a las potencias **2** y **3**. De esta manera, la función sería más general, reutilizable y podría emplearse en diferentes programas sin necesidad de modificar su código.

Ejercicio 61: Potencia con parámetros

```
#include <stdio.h>
#include <math.h>

// Función que calcula la potencia
float potencia(float numero, int exponente)
{
    return pow(numero, exponente);
}

// Función para imprimir el resultado
void imprimir(float resultado, int exponente)
{
    printf("\nEl resultado de elevar el numero a la potencia %d es: %.2f\n", exponente,
resultado);
}

int main()
{
    float n1, n2, numero, resultado;
    int opcion, pot;

    printf("Ingrese el primer numero: ");
    scanf("%f", &n1);

    printf("Ingrese el segundo numero: ");
    scanf("%f", &n2);

    printf("\nSeleccione el numero que desea potenciar:\n");
    printf("1. Primer numero\n");
```

```

printf("2. Segundo numero\n");
printf("Opcion: ");
scanf("%d", &opcion);

if(opcion == 1)
    numero = n1;
else if(opcion == 2)
    numero = n2;
else

```

```

"C:\Users\LAPTOP HP\Docum X + v
Ingrese el primer numero: 4
Ingrese el segundo numero: 3

Seleccione el numero que desea potenciar:
1. Primer numero
2. Segundo numero
Opcion: 1
Ingrese la potencia (2 o 3): 2

El resultado de elevar el numero a la potencia 2 es: 16.00

Process returned 0 (0x0)   execution time : 28.730 s
Press any key to continue.

```

```

"C:\Users\LAPTOP HP\Docum X + v
Ingrese el primer numero: 4
Ingrese el segundo numero: 3

Seleccione el numero que desea potenciar:
1. Primer numero
2. Segundo numero
Opcion: 2
Ingrese la potencia (2 o 3): 3

El resultado de elevar el numero a la potencia 3 es: 27.00

Process returned 0 (0x0)   execution time : 7.091 s
Press any key to continue.
|

```

```

{
    printf("Opcion invalida.\n");
    return 0;
}

printf("Ingrese la potencia (2 o 3): ");
scanf("%d", &pot);

if(pot == 2 || pot == 3)
{
    resultado = potencia(numero, pot);
    imprimir(resultado, pot);
}
else
{
    printf("Solo se permiten las potencias 2 y 3.\n");
}

return 0;
}

```

Pregunta de análisis: ¿Qué mejora permitiría que la función potencia() sea más general y reutilizable? sería más general y reutilizable si permitiera calcular cualquier exponente que ingrese el usuario, en lugar de limitarse solo a las potencias 2 y 3

4. Conclusiones

- El uso de funciones permite organizar mejor el programa, haciendo que el código sea más claro, ordenado y fácil de mantener.
- Las funciones con parámetros y retorno de valores facilitan la reutilización del código, evitando repetir instrucciones y mejorando la eficiencia del programa.

5. Recomendaciones

- Validar siempre los datos ingresados por el usuario para evitar errores durante la ejecución del programa.
- Utilizar nombres descriptivos para las funciones y variables, de modo que el código sea más fácil de comprender y mantener.

Archivos .cpp



ejercicio 60.c



Untitled1.c



Ejer 58.c



EJER 61.c